

A linear-time construction of the relative neighborhood graph from the Delaunay triangulation

Andrzej Lingas

Department of Computer Science, Lund University, Box 118, 221 00 Lund, Sweden

Communicated by Subhash Suri; submitted 8 October 1992; accepted 11 January 1994

Abstract

A very simple linear-time algorithm for constructing the relative neighborhood graph $\text{RNG}(V)$ for a finite set V of points in the plane from the Delaunay triangulation of V is presented. It is extended to include the construction of the so called β -skeletons (generalization of $\text{RNG}(V)$) in the spectrum $1 \leq \beta \leq 2$ in linear time from the Delaunay triangulation under the metric L_p for $1 < p < \infty$ if $\beta = 2$ or $p = 2$.

Key words: Computational geometry; Pattern recognition; Delaunay triangulation; Relative neighborhood graph; β -Skeleton, Gabriel graph

1. Introduction

Toussaint has introduced the notion of relative neighborhood graph and shown its applications in pattern recognition [10]. Several other authors studied the problem of efficiently constructing the graph and its generalizations [1, 3, 4, 5, 12].

Consider a finite set V of points in the plane. For a, b in V , let $\text{lune}(a, b)$ be the set of points in the plane whose Euclidean distance to each of a and b is less than the Euclidean distance between a and b . The edge (a, b) is an edge of the relative neighborhood graph on V , $\text{RNG}(V)$, if and only if $\text{lune}(a, b)$ is free from points in V .

$\text{RNG}(V)$ is shown to be a subgraph of the Delaunay triangulation of V , $\text{DT}(V)$ [9] (see Fig. 1). Supowit has used this fact to design the first efficient algorithm for $\text{RNG}(V)$. His algorithm takes $O(n \log n)$ -time to construct $\text{RNG}(V)$ from $\text{DT}(V)$ [12]. This combined with any of the known $O(n \log n)$ -time algorithms for $\text{DT}(V)$ yields an $O(n \log n)$ -time algorithm for $\text{RNG}(V)$ which is asymptotically optimal [12].

Jaromczyk and Kowaluk have later shown that $\text{RNG}(V)$ can be constructed from $\text{DT}(V)$ in time $O(n\alpha(n, n))$, where $\alpha(\cdot)$ is the inverse of the Ackermans function [3].

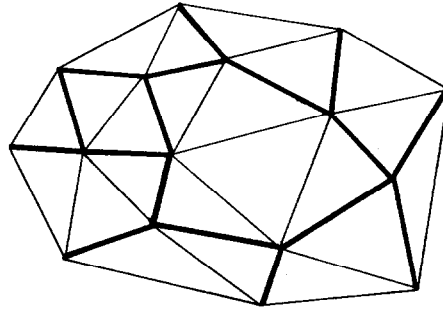


Fig. 1. $\text{RNG}(V)$ (fat lines) is a subgraph of $\text{DT}(V)$.

The time performance of their algorithm is achieved due to natural applications of UNION-FIND. Recently, the two above authors together with F.F. Yao have applied the static variant of UNION-FIND due to Gabow and Tarjan [2] instead, reducing the time-performance of their algorithm to a linear one [4]. They have also generalized their algorithm to construct the so called β -skeleton (which is the Gabriel Graph for $\beta = 1$ and the Relative Neighborhood Graph for $\beta = 2$, see [4, 6]) in the spectrum $1 \leq \beta \leq 2$ in linear time from the Delaunay triangulation $\text{DT}_p(V)$ under the metric L_p , $1 < p < \infty$.

This paper shows that $\text{RNG}(V)$ can be constructed from $\text{DT}(V)$ by a very simple linear-time (and linear-space) algorithm which does not use any advanced data structure (Section 2, 4). The same method is applied to construct any β -skeleton in the spectrum $1 \leq \beta \leq 2$ under the metric L_p for $1 < p < \infty$, where $\beta = 2$ or $p = 2$, in linear time from $\text{DT}_p(V)$ (Section 5). Other directions of generalization are also discussed (Section 5).

2. The algorithm

We shall use the following notation throughout this section and the rest of the paper. For two points a and b in the plane (a, b) denotes the edge (i.e., the straight-line segment) joining them. For an edge $e = (a, b)$ $\text{lune}(e)$ stands for $\text{lune}(a, b)$. Next, for a point v and a direction k , $\text{ray}(v, k)$ denotes the ray whose endpoint is v and which is infinite in the direction k . The cardinality of a finite set S is denoted by $|S|$.

2.1. Main ideas

The first main idea in our algorithm is the reduction of the problem to a simple polygon case. It relies on a lemma due to Supowit (Lemma 3 in [12]). To describe Supowit's lemma we need the following notation. Let v be a point in the plane, and let e and e_1 be two straight-line segments in the plane which are disjoint from v . We say that e_1 blocks e from v in direction k if $\text{ray}(v, k)$ intersects the interior of e_1 before e .

Lemma 2.1 [12]. *Let e be an edge in $\text{DT}(V) - \text{RNG}(V)$. There exists a point $v \in \text{lune}(e) \cap V$ and a direction k such that for each edge $e_1 \in \text{DT}(V)$, if e_1 blocks v from e in direction k then $v \in \text{lune}(e_1)$.*

The following corollary is immediate.

Corollary 2.2. *Let e be an edge in $\text{DT}(V) - \text{RNG}(V)$ and let H be a subgraph of $\text{RNG}(V)$. There exists a point $v \in \text{lune}(e) \cap V$ and a direction k such that v is not blocked from e by any edge of H in direction k , and if an edge e_1 in $\text{DT}(V) - H$ blocks v from e in direction k then $v \in \text{lune}(e_1)$.*

The Euclidean minimum spanning tree of V , $\text{EMST}(V)$, is a subgraph of $\text{RNG}(V)$ [10]. Partition the area of the convex hull of V into simple polygons with possible double edges by drawing the boundary $\text{CH}(V)$ of the convex hull and $\text{EMST}(V)$ (see Fig. 2). Note that the edges of the input Delaunay triangulation of V , $\text{DT}(V)$, that don't belong to $\text{CH}(V) \cup \text{EMST}(V)$ form triangulations of the polygons. Substitute $\text{EMST}(V)$ for H in Corollary 2.2. Then, for such a vertex v , ray(v, k) up to its intersection with e totally lies within one of the simple polygons induced by $\text{CH}(V) \cup \text{EMST}(V)$. Therefore, it is sufficient to consider only the vertices of the polygon $P(e)$ that e lies in while looking for witnesses of $e \notin \text{RNG}(V)$. Hence, we obtain the following lemma reducing the problem of constructing $\text{RNG}(V)$ to a simple polygon case.

Lemma 2.3. *Let e be an edge in $\text{DT}(V) - \text{RNG}(V)$. Then there exists a vertex v of $P(e)$ in $\text{lune}(e)$ and a direction k such that ray(v, k) intersects e and lies within $P(e)$ at least to the intersection point, and if an edge e_1 in $\text{DT}(V) - (\text{CH}(V) \cup \text{EMST}(V))$ blocks v from e in direction k then $v \in \text{lune}(e_1)$.*

The further ideas used in our algorithm concern an efficient construction of sets of potential witnesses of $e \notin \text{RNG}(V)$ for all edges e in $\text{DT}(V) - \text{EMST}(V)$. We begin with defining their subsets $w_1(e)$, $w_2(e)$ consisting of true witnesses.

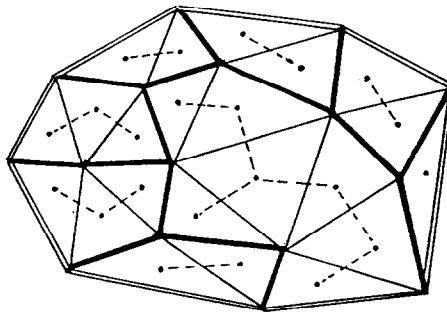


Fig. 2. The partition into triangulated polygons induced by $\text{DT}(V)$, $\text{CH}(V)$ (double lines) and $\text{EMST}(V)$ (fat lines). The trees dual to the triangulated polygons are marked with broken lines.

For each polygon induced by $\text{CH}(V) \cup \text{EMST}(V)$, we distinguish an edge in $\text{CH}(V)$ on the perimeter of $P(e)$ (there exist at least one such edge). Let $P_1(e)$, $P_2(e)$ be the subpolygons the edge e splits $P(e)$ into such that the distinguished edge lies within $P_2(e)$ (if e is the distinguished edge then $P_2(e)$ is simply the degenerate polygon consisting of e). For $i = 1, 2$, let $w_i(e)$ be the set of vertices v of $P_i(e)$ in $\text{lune}(e)$ such that there is a direction k in which $\text{ray}(v, k)$ intersects e and lies within $P_i(e)$ at least to the intersection point.

By Lemma 2.3 and the above definitions, we obtain the following corollary immediately.

Corollary 2.4. *An edge e in $\text{DT}(V) - \text{EMST}(V)$ is in RNG if and only if $w_1(e) \cup w_2(e) = \emptyset$.*

For $i = 1, 2$, our algorithm will compute a superset $pw_i(e)$ of $w_i(e)$ such that $pw_i(e) = \emptyset$ if and only if $w_i(e) = \emptyset$. The computation of the supersets (i.e., sets of potential witnesses of $e \notin \text{RNG}(V)$) relies on a lemma due to Jaromczyk, Kowaluk and Yao from [4]. To present this lemma we need the following notation.

Let $v \in V$, and let e be an edge in $\text{DT}(V)$. We say that v eliminates e if v is in $\text{lune}(e)$. Consider a triangle (a, b, c) in $\text{DT}(V)$. Suppose that v eliminates (a, b) . We say that v is external to (a, b, c) with respect to (a, b) if and only if c and v are separated by the straight-line passing through (a, b) .

Lemma 2.5. [3]. *Let $v, w \in V$ eliminate an edge (a, b) of a triangle (a, b, c) in $\text{DT}(V)$. If v and w are external to (a, b, c) with respect to (a, b) then (a, c) and (b, c) cannot be both eliminated by v and w .*

The following immediate corollary will be especially useful.

Corollary 2.6. *Let (a, b, c) be a triangle in $\text{DT}(V)$, and let W be a set of points in $V \cap \text{lune}(a, b)$ external to (a, b, c) with respect to (a, b) . Then either $W \cap \text{lune}(a, c) = \emptyset$ or $W \cap \text{lune}(b, c) = \emptyset$.*

2.2. Description

We shall call our algorithm for $\text{RNG}(V)$ presented in this subsection Algorithm 1.

Algorithm 1 starts from constructing $\text{CH}(V) \cup \text{EMST}(V)$ and the polygons induced by $\text{CH}(V) \cup \text{EMST}(V)$ (see Fig. 2). Next it constructs the trees dual to the triangulations of the polygons yielded by $\text{DT}(V) - (\text{CH}(V) \cup \text{EMST}(V))$. It also roots each of the trees at the node corresponding to the triangle containing the distinguished convex hull edge. After these preprocessing steps, Algorithm 1 computes the sets $pw_1(e)$, $pw_2(e)$ for all $e \in \text{DT}(V) - \text{EMST}(V)$ in two traversals of the trees. Next, it outputs all the edges $e \in \text{DT}(V) - \text{EMST}(V)$ for which $pw_1(e) \cup pw_2(e) = \emptyset$ and all the edges of $\text{EMST}(V)$ as the edges of $\text{RNG}(V)$.

2.2.1. The first traversal

The first traversal is in postorder. During this traversal, the sets $pw_1(e)$ and preliminary sets $pw_2(e)$ are determined for all e in $DT(V) - EMST(V)$. Let (a, b, c) be the triangle corresponding to the tree node currently scanned where for each e in $\{(a, c), (b, c)\}$, either e lies on the perimeter of the polygon dual to the tree or $pw_1(e)$ is already determined. In the former case, Algorithm 1 sets $pw_1(e)$ to an empty set. Next, it tests c for containment in $\text{lune}(a, b)$ and if the test is positive it sets $pw_1(a, b)$ to $\{c\}$.

Comment: If a vertex v of $P_1(a, c)$ or $P_1(b, c)$ is weakly visible from (a, b) in $P(a, b)$ and satisfies Lemma 2.3 for $e = (a, b)$ then $v \in w_1(a, c) \cup w_1(b, c)$. Consequently, the set $pw_1(a, b)$ can be restricted to $\{c\} \cup pw_1(a, c) \cup pw_1(b, c)$. Also, if the tree node corresponding to the triangle (a, b, c) has degree three then some members in $pw_1(a, c)$ may belong to $w_2(b, c)$, i.e., be witnesses of $(b, c) \notin \text{RNG}(V)$ coming from $P_2(b, c)$ (see Fig. 3). Symmetrically, $pw_1(b, c)$ can contribute to $w_2(a, c)$. Therefore, Algorithm 1 prunes and divides the set $pw_1(a, c) \cup pw_1(b, c)$ between $pw_1(a, b)$, $pw_2(b, c)$ and $pw_2(a, c)$ by Corollary 2.6 as follows.

It sets W to $pw_1(a, c)$ and executes the following block.

Pick an element w in W . If w is in $\text{lune}(a, b)$ then augment $pw_1(a, b)$ by W . Otherwise, if w is in $\text{lune}(b, c)$ then set $pw_2(b, c)$ to W . If none of the two cases holds delete w from W and repeat the block if W is not empty.

After that Algorithm 1 sets W to $pw_1(b, c)$ and executes the symmetrical block where the roles of (a, c) and (b, c) are exchanged.

2.2.2. The second traversal

The second traversal is in preorder. During this traversal, the sets $pw_2(e)$ are finally determined for all e in $DT(V) - EMST(V)$. Let (a, b, c) be the triangle corresponding to the tree node currently scanned where either (a, b) lies on the perimeter of the polygon dual to the tree or $pw_2(a, b)$ is already determined. In the former case, Algorithm 1 sets $pw_2(a, b)$ to an empty set. Next, it tests whether b and a are

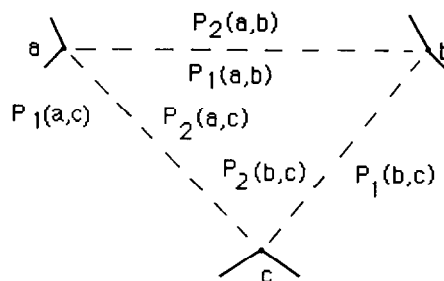


Fig. 3. An example of the triangle (a, b, c) and the subpolygons induced by its edges.

respectively in $\text{lune}(a, c)$ and $\text{lune}(b, c)$ and if so $pw_2(a, c)$ is set to $\{b\} \cup pw_2(a, c)$ and/or $pw_2(b, c)$ is set to $\{a\} \cup pw_2(b, c)$ respectively.

Comment: If a vertex of $P_2(a, b)$ is weakly visible from (a, c) or (b, c) in the dual polygon and satisfies Lemma 2.3 for $e = (a, c)$ or $e = (b, c)$ then v is in $w_2(a, b)$ (see Fig. 3). This restricts the set $pw_2(a, c)$ of potential witnesses of $(a, c) \notin \text{RNG}(V)$ in $P_2(a, c)$ to $pw_2(a, b) \cup \{b\}$ plus the preliminary set $pw_2(a, c)$. Analogously $pw_2(b, c)$ can be restricted to $pw_2(a, b) \cup \{a\}$ plus the preliminary $pw_2(b, c)$ set. Relying on Corollary 2.6, Algorithm 1 prunes the restriction sets as follows.

It sets W to $pw_2(a, b)$ and executes the following block.

Test the first element w in W for the containment in $\text{lune}(a, c)$ and/or $\text{lune}(b, c)$ appropriately. If w is in $\text{lune}(a, c)$ then set $pw_2(a, c)$ to $pw_2(a, c) \cup W$. If w is in $\text{lune}(b, c)$ then set $pw_2(b, c)$ to $pw_2(b, c) \cup W$. If none of the two cases hold delete w from W and repeat the block if W is not empty.

2.3. Analysis

We begin with the proof of correctness of Algorithm 1.

Lemma 2.7. *For each edge e in $\text{DT}(V) - \text{EMST}(V)$, both $pw_1(e)$ and $pw_2(e)$ are empty after the two traversals in Algorithm 1 if and only if e is in $\text{RNG}(V)$.*

Proof. If e is in $\text{RNG}(V)$ then no vertex v that belongs to $\text{lune}(e)$ can be found in the two traversals. Hence, both $pw_1(e)$ and $pw_2(e)$ are always empty by the traversal procedures.

The proof of the reverse implication is as follows. Let $v \in V \cap \text{lune}(e)$ and direction k satisfy Lemma 2.3 with respect to e . Let t be a triangle in the polygon $P(e)$ such that e is an edge of t . Consider the rooted tree dual to $P(e)$ (see Fig. 2).

Case 1: There exists a triangle t_v in $\text{DT}(V)$ cornered by v which includes an initial fragment of $\text{ray}(v, k)$ and corresponds to a node in the subtree of T rooted at t .

Let P be the path from t_v to t in the subtree. By Lemma 2.3, $\text{ray}(v, k)$ intersects all edges e_1 between the neighboring triangles on the path P . Also, for all such edges e_1 , $v \in \text{lune}(e_1)$ holds. Hence, v is in $pw_1(e_1)$ inductively by the first traversal procedure and Corollary 2.6. Consequently, v is in $pw_1(e)$.

Case 2: There exists a triangle t_v cornered by v such that the t_v includes an initial fragment of $\text{ray}(v, k)$ and the subtree of T rooted at the node corresponding to t_v includes the node corresponding to t .

By Lemma 2.3, Corollary 2.6 and the second traversal procedure, we can inductively prove $v \in pw_2(e)$, analogously as in Case 1.

Case 3: Neither Case 1 nor Case 2 hold.

Let s be the lowest common ancestor in T of t and any triangle t_v cornered by v which includes an initial fragment of $\text{ray}(v, k)$. Let s_t be the edge of the triangle s on

the side of t and let s_v be the edge of s on the side of t_v . By Lemma 2.3, $\text{ray}(v, k)$ intersects s_v and s_t on its way to e , and the inclusions $v \in \text{lune}(s_v)$, $v \in \text{lune}(s_t)$ hold. By Case 1, we have also $v \in pw_1(s_v)$. Hence, v is in (preliminary) $pw_2(s_t)$ after the first traversal. Next, arguing analogously as in the previous cases, we obtain $v \in pw_2(e)$ by induction using Lemma 2.3, Corollary 2.6 and the second traversal procedure. $\square$

The next lemma analyses the time complexity of Algorithm 1.

Lemma 2.8. *Algorithm 1 runs in linear time.*

Proof. Given $\text{DT}(V)$, we can construct $\text{EMST}(V)$ in linear time [9]. Clearly, also $\text{CH}(V)$, the polygons and the dual trees can be easily determined in linear time then. Thus, the preprocessing steps take linear time.

To execute the two traversal procedures, we implement the sets $pw_i(\)$ as lists. Therefore, the operations of union for two such sets or of augmenting such a set with a single element, or testing for non-emptiness take constant time. Also, before the substitution of such a set $pw_i(\)$ for W we can test it for non-emptiness so we don't need to store it any longer. Therefore we can use the original list representation of such a set for the operations on W . The total number of the triangles (a, b, c) considered in the two traversals is clearly linear in the input size. The time taken by the traversal procedures at the node corresponding to such a triangle is $O(1)$ time plus time proportional to the number of deletions from W . Therefore, it is sufficient to estimate the total number of deletions from W during the performance of Algorithm 1.

First, consider a deletion of an element w from W during the first, postorder traversal. Note that w cannot occur in $pw_1(a, b)$ then. Since w is a vertex of some triangle that is a descendant of the triangle (a, b, c) in the tree traversed, it can never be inserted again in any pw_1 set during this tree traversal. Hence, w can never occur again in W , and consequently, the total number of deletions from W is bounded by the total number of different vertices that can be in W during this tree traversal. The latter number is bounded by the total number of vertices in the polygon dual to the tree.

It remains to consider the case of a deletion of an element w from W during the second, preorder traversal. After the deletion, w cannot occur in $pw_2(a, c)$ or $pw_2(b, c)$ even if the deleted w is a member of a preliminary pw_2 set. In the latter case, w can never again occur in any preliminary pw_2 set by the disjointness of these sets. Otherwise, w is a vertex of a triangle that is an ancestor of the triangle (a, b, c) in the tree and therefore w can never again be inserted in any pw_2 set during this traversal. It follows that each vertex of the polygon dual to the tree traversed can be deleted from W at most twice.

Consequently, the total time taken by the preorder traversal is also linear in the polygon size. Summarizing, the total number of deletions from W during the performance of Algorithm 1 is linear in the input size.

Thus, the two traversal procedures also take linear time. As the listing of the edges of $\text{EMST}(V)$ and the edges $e \in \text{DT}(V) - \text{EMST}(V)$ for which both $pw_1(e)$ and $pw_2(e)$

are empty can be done trivially in linear time, the whole algorithm takes linear time. $\square$

Combining Lemma 2.7 with Lemma 2.8 we obtain our main result.

Theorem 2.9. *Algorithm 1 computes $\text{RNG}(V)$ from $\text{DT}(V)$ in linear time.*

3. Extensions

We can generalize our linear-time algorithm for $\text{RNG}(V)$ to include the construction of the so called β -skeleton for V , $1 \leq \beta \leq 2$, under the L_p -metric for $1 < p < \infty$ if $\beta = 2$ or $p = 2$ (see [4, 6]). The distance function d_p in the L_p metric in the two-dimensional plane is defined by:

$$d_p(q, r) = (|X(q) - X(r)|^p + |Y(q) - Y(r)|^p)^{\frac{1}{p}}$$

For $1 \leq \beta \leq 2$, and a, b in V , $\text{lune}_{\beta, p}(a, b)$ is the interior of the intersection of two L_p circles, with radii equal to $\frac{1}{2}\beta d_p(a, b)$, centered on (a, b) in the distance $\frac{1}{2}\beta d_p(a, b)$ from a or b respectively. Now, the edge (a, b) is in the β -skeleton $G_{\beta, p}(V)$ for V in the L_p metric if and only if $\text{lune}_{\beta, p}(a, b)$ is free from vertices in V .

Note that $G_{2,2}(V) = \text{RNG}(V)$. Also, $G_{1,2}(V)$ is well known in the literature as the Gabriel Graph of V ($\text{GG}(V)$ for short) [8]. $\text{GG}(V)$ is known to be a subset of $\text{DT}(V)$ (see [8]) which implies that the $G_{\beta, 2}$ is a subset of $\text{DT}(V)$ for any β in the range $[1, 2]$. In fact, a linear-time algorithm for constructing $\text{GG}(V)$ from $\text{DT}(V)$ using a different approach has been known for a long time in the literature [8].

The Delaunay triangulation $\text{DT}_p(V)$ of V in the L_p metric can be defined as the straight-line dual to the Voronoi diagram in the L_p metric space (under the assumption that no four points in V are co-circular). Let $\text{MST}_p(V)$ denote any minimum spanning tree on V in the L_p metric space. The following general lemmata immediately follow from corresponding lemmata proved by Jaromczyk et al. in [4].

Lemma 3.1 (see Lemma 2.2 in [4]). *For $1 < p < \infty$ and $1 \leq \beta \leq 2$, $\text{MST}_p(V) \subset G_{\beta, p} \subset \text{DT}_p(V)$.*

Note that the above lemma enables us to obtain $G_{\beta, p}$ by pruning $\text{DT}_p(V)$. Also, this lemma together with the next one which is a non-trivial generalization of Lemma 2.1 makes it possible to perform the reduction to the polygon case analogous to that in Algorithm 1.

Lemma 3.2 (see Lemma 3.4 in [4]). *Let $1 < p < \infty$, $1 \leq \beta \leq 2$, where $p = 2$ or $\beta = 2$, and let e be in $\text{DT}(V) - G_{\beta, p}(V)$. There is a point v in V and a direction k such that for each edge $e_1 \in \text{DT}_p(V)$, if e_1 blocks v from e in direction k then $v \in \text{lune}(e_1)$.*

The last lemma both strengthens and generalizes Lemma 2.5.

Lemma 3.3 (see Lemma 3.3 in [4]). *Let $1 < p < \infty$, $1 \leq \beta \leq 2$, and let (a, b, c) be a triangle in $DT_p(V)$. If v eliminates (a, b) then v can eliminate only the (strictly) longer of (a, c) and (b, c) .*

Let Algorithm 2 denote the algorithm resulting from substituting $DT_p(V)$ for $DT(V)$, $MST_p(V)$ for $EMST(V)$, $G_{\beta,p}(V)$ for $RNG(V)$, $\text{lune}_{\beta,p}(\cdot)$ for $\text{lune}(\cdot)$ etc. in Algorithm 1. Analogously as the correctness of Algorithm 1 follows from the inclusions $EMST(V) \subset RNG(V) \subset DT(V)$ the Lemmata 2.1, 2.5, the correctness of Algorithm 2 follows from Lemmata 3.1, 3.2, 3.3 (under the assumption that no four points in V are co- L_p -circular which implies that $DT_p(V)$ is a complete triangulation [7]). Hence, skipping here the technical details of extending Algorithm 2 to handle the degenerate case, we obtain the following theorem.

Theorem 3.4. *For $1 < p < \infty$, $1 \leq \beta \leq 2$, where $p = 2$ or $\beta = 2$, Algorithm 2 computes $G_{\beta,p}$ from $DT_p(V)$ in linear time.*

Finally, we could also extend our method to build the so called constrained RNG for a planar straight-line graph G in linear-time from the constrained Delaunay triangulation of G [11]. An edge (a, b) is in $RNG(G)$ if it does not properly intersect any edge in G and $\text{lune}(a, b)$ is free from vertices of G visible both from a and b in the presence of the edges of G treated as visibility barriers. Su and Chang generalized Supowit's $O(n \log n)$ -time algorithm for $RNG(V)$ to produce $RNG(G)$ [11].

Acknowledgements

The author is very grateful to unknown referees and Jerzy Jaromczyk for their careful reading and insightful comments on this work.

References

- [1] P.K. Agarwal and J. Matousek, Relative neighborhood graphs in three dimensions, *Comput. Geom. Theory Appl.* 2 (1992) 1–14.
- [2] H.N. Gabow and R.E. Tarjan, A linear-time algorithm for a special case of disjoint set union, *J. Comput. Sys. Sci.* 30 (1985) 209–221.
- [3] J.W. Jaromczyk and M. Kowaluk, A note on relative neighborhood graphs, *Proc. 3rd ACM Symp. on Computational Geometry* (1987) 233–241.
- [4] J.W. Jaromczyk and M. Kowaluk and F.F. Yao, An optimal algorithm for constructing β -skeletons in L_p metric, *SIAM J. Comput.*, to appear.
- [5] J.W. Jaromczyk and G.T. Toussaint, Relative neighborhood graphs and their relatives, *Proceedings of the IEEE* 80(9) (1992) 1502–1516.
- [6] D.G. Kirkpatrick and J.D. Radke, A framework for computational morphology, in: G. Toussaint, ed., *Computational Geometry* (North-Holland, Amsterdam, 1985) 217–248.

- [7] D.T. Lee and B. Schachter, Two algorithms for constructing Delaunay triangulations, *Int. J. Comput. Info. Sci.* 9 (1980) 219–242.
- [8] D.W. Matula and R.R. Sokal, Properties of Gabriel graphs relevant to geographic variation search and the clustering of points in the plane, *Geogr. Analysis* 12 (1980) 205–222.
- [9] F.P. Preparata and M.I. Shamos, *Computational Geometry: An Introduction*, Texts and Monographs in Theoretical Computer Science (Springer, New York, 1985).
- [10] G.T. Toussaint, The relative neighborhood graph of a finite planar set, *Pattern Recognition* 12 (1980) 261–268.
- [11] T.H. Su and R.C. Chang, Computing the constrained relative neighborhood graphs and constrained Gabriel graphs in Euclidean plane, *Pattern Recognition* 24 (1991) 221–230.
- [12] K.J. Supowit, The relative neighborhood graph with an application to minimum spanning trees, *J. Assoc. Comput. Mach.* 30 (1983) 428–448.